信 息 工 程 学 院

《深度学习》实验报告

学院：信息工程学院　**班级**：数科 2102　**姓名**：陈　**学号**:202101　**成绩**:

一、实验题目

车辆识别

人脸识别与神经风格迁移

NLP 词向量的运算与 Emoji 生成器

机器翻译和触发词检测

二、实验目的

使用 YOLO 模型进行识别和定位车辆

完成 Happy House 的人脸识别

完成神经风格转换

三、实验平台

希冀平台

Jupyter

四、实验过程及步骤

车辆识别

1. 利用种类分值门槛进行过滤

实验报告的内容与格式按任课教师的要求书写。

In [4]:
```python
# GRADED FUNCTION: yolo_filter_boxes

def yolo_filter_boxes(box_confidence, boxes, box_class_probs, threshold = .6):
    """通过对对象和类的置信度设置阈值来过滤YOLO框。

    参数:
    box_confidence -- tensor of shape (19, 19, 5, 1)
    boxes -- tensor of shape (19, 19, 5, 4)
    box_class_probs -- tensor of shape (19, 19, 5, 80)
    threshold -- real value, if [ highest class probability score < threshold], then get rid of the corresponding box

    返回值:
    scores -- tensor of shape (None,), containing the class probability score for selected boxes
    boxes -- tensor of shape (None, 4), containing (b_x, b_y, b_h, b_w) coordinates of selected boxes
    classes -- tensor of shape (None,), containing the index of the class detected by the selected boxes

    Note: "None" is here because you don't know the exact number of selected boxes, as it depends on the threshold.
    For example, the actual output size of scores would be (10,) if there are 10 boxes.
    """

    # Step 1: Compute box scores
    ### START CODE HERE ### (≈ 1 line)
    box_scores = box_confidence * box_class_probs
    ### END CODE HERE ###

    # Step 2: 通过最大的box_scores查找box_classes，跟踪相应得分
    ### START CODE HERE ### (≈ 2 lines)
    box_classes = K.argmax(box_scores, axis=-1)
    box_class_scores = K.max(box_scores, axis=-1, keepdims=False)
    ### END CODE HERE ###

    # Step 3: 使用“阈值”基于“ box_class_scores”创建过滤掩码。 遮罩的尺寸应与box_class_scores相同，并且对于要保留的框为True（概率>
    ### START CODE HERE ### (≈ 1 line)
    filtering_mask = box_class_scores >= threshold
    ### END CODE HERE ###

    # Step 4: Apply the mask to scores, boxes and classes
    ### START CODE HERE ### (≈ 3 lines)
    scores = tf.boolean_mask(box_class_scores, filtering_mask)
    boxes = tf.boolean_mask(boxes, filtering_mask)
    classes = tf.boolean_mask(box_classes, filtering_mask)
    ### END CODE HERE ###
```

In [5]:
```python
#测试yolo_filter_boxes
with tf.Session() as test_a:
    box_confidence = tf.random_normal([19, 19, 5, 1], mean=1, stddev=4, seed = 1)
    boxes = tf.random_normal([19, 19, 5, 4], mean=1, stddev=4, seed = 1)
    box_class_probs = tf.random_normal([19, 19, 5, 80], mean=1, stddev=4, seed = 1)
    scores, boxes, classes = yolo_filter_boxes(box_confidence, boxes, box_class_probs, threshold = 0.5)
    print("scores[2] = " + str(scores[2].eval()))
    print("boxes[2] = " + str(boxes[2].eval()))
    print("classes[2] = " + str(classes[2].eval()))
    print("scores.shape = " + str(scores.shape))
    print("boxes.shape = " + str(boxes.shape))
    print("classes.shape = " + str(classes.shape))
```

```
WARNING: Logging before flag parsing goes to stderr.
W1130 06:12:40.448324 139728068544320 deprecation.py:323] From /opt/conda/lib/python3.6/site-packages/tensorflow_core/python/ops/ar
ray_ops.py:1475: where (from tensorflow.python.ops.array_ops) is deprecated and will be removed in a future version.
Instructions for updating:
Use tf.where in 2.0, which has the same broadcast rule as np.where
```

```
scores[2] = 10.750582
boxes[2] = [ 8.426533   3.2713668 -0.5313436 -4.9413733]
classes[2] = 7
scores.shape = (?,)
boxes.shape = (?, 4)
classes.shape = (?,)
```

2、非最大抑制

In [6]:
```
# GRADED FUNCTION: iou

def iou(box1, box2):
    """在box1和box2之间实现联合交叉（IoU）

    参数:
    box1 -- first box, list object with coordinates (x1, y1, x2, y2)
    box2 -- second box, list object with coordinates (x1, y1, x2, y2)
    """

    # 计算box1和box2相交的（y1, x1, y2, x2）坐标。 计算其面积。
    ### START CODE HERE ### (≈ 5 lines)
    xi1 = np.maximum(box1[0], box2[0])
    yi1 = np.maximum(box1[1], box2[1])
    xi2 = np.minimum(box1[2], box2[2])
    yi2 = np.minimum(box1[3], box2[3])
    inter_area = (xi1-xi2)*(yi1-yi2)
    ### END CODE HERE ###

    # 通过使用公式计算联合面积：Union（A, B）= A + B-Inter（A, B）
    ### START CODE HERE ### (≈ 3 lines)
    box1_area = (box1[2]-box1[0])*(box1[3]-box1[1])
    box2_area = (box2[2]-box2[0])*(box2[3]-box2[1])
    union_area = box1_area + box2_area - inter_area
    ### END CODE HERE ###

    # 计算IoU
    ### START CODE HERE ### (≈ 1 line)
    iou = inter_area / union_area
    ### END CODE HERE ###

    return iou
```

In [7]:
```
#测试iou
box1 = (2, 1, 4, 3)
box2 = (1, 2, 3, 4)
print("iou = " + str(iou(box1, box2)))
```

```
iou = 0.14285714285714285
```

练习：使用 TensorFlow 实现 yolo_non_max_suppression()

In [8]:
```
# GRADED FUNCTION: yolo_non_max_suppression

def yolo_non_max_suppression(scores, boxes, classes, max_boxes = 10, iou_threshold = 0.5):
    """
    对一组方框应用非最大抑制(NMS)

    参数:
    scores -- tensor of shape (None,), output of yolo_filter_boxes()
    boxes -- tensor of shape (None, 4), output of yolo_filter_boxes() that have been scaled to the image size (see later)
    classes -- tensor of shape (None,), output of yolo_filter_boxes()
    max_boxes -- integer, maximum number of predicted boxes you'd like
    iou_threshold -- real value, "intersection over union" threshold used for NMS filtering

    返回值:
    scores -- tensor of shape (, None), predicted score for each box
    boxes -- tensor of shape (4, None), predicted box coordinates
    classes -- tensor of shape (, None), predicted class for each box

    Note: The "None" dimension of the output tensors has obviously to be less than max_boxes. Note also that this
    function will transpose the shapes of scores, boxes, classes. This is made for convenience.
    """

    max_boxes_tensor = K.variable(max_boxes, dtype='int32')     # tensor to be used in tf.image.non_max_suppression()
    K.get_session().run(tf.variables_initializer([max_boxes_tensor])) # initialize variable max_boxes_tensor

    # 使用tf.image.non_max_suppression（）获取与您保留的盒子相对应的索引列表
    ### START CODE HERE ### (≈ 1 line)
    nms_indices = tf.image.non_max_suppression(boxes, scores,max_boxes,iou_threshold)
    ### END CODE HERE ###

    # 使用K.gather（）从成绩，方框和类别中仅选择nms_indices
    ### START CODE HERE ### (≈ 3 lines)
    scores = K.gather(scores, nms_indices)
    boxes = K.gather(boxes, nms_indices)
    classes = K.gather(classes, nms_indices)
    ### END CODE HERE ###

    return scores, boxes, classes
```

In [9]:

```python
#测试yolo_non_max_suppression

with tf.Session() as test_b:
    scores = tf.random_normal([54,], mean=1, stddev=4, seed = 1)
    boxes = tf.random_normal([54, 4], mean=1, stddev=4, seed = 1)
    classes = tf.random_normal([54,], mean=1, stddev=4, seed = 1)
    scores, boxes, classes = yolo_non_max_suppression(scores, boxes, classes)
    print("scores[2] = " + str(scores[2].eval()))
    print("boxes[2] = " + str(boxes[2].eval()))
    print("classes[2] = " + str(classes[2].eval()))
    print("scores.shape = " + str(scores.eval().shape))
    print("boxes.shape = " + str(boxes.eval().shape))
    print("classes.shape = " + str(classes.eval().shape))
```

```
W1130 06:12:40.587315 139728068544320 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_b
ackend.py:174: The name tf.get_default_session is deprecated. Please use tf.compat.v1.get_default_session instead.

W1130 06:12:40.588238 139728068544320 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_b
ackend.py:190: The name tf.global_variables is deprecated. Please use tf.compat.v1.global_variables instead.

W1130 06:12:40.589269 139728068544320 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_b
ackend.py:199: The name tf.is_variable_initialized is deprecated. Please use tf.compat.v1.is_variable_initialized instead.

W1130 06:12:40.596479 139728068544320 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_b
ackend.py:206: The name tf.variables_initializer is deprecated. Please use tf.compat.v1.variables_initializer instead.
```

```
scores[2] = 6.938395
boxes[2] = [-5.299932    3.1379814   4.450367    0.95942086]
classes[2] = -2.2452729
scores.shape = (10,)
boxes.shape = (10, 4)
classes.shape = (10,)
```

4、包装过滤器

```python
def yolo_eval(yolo_outputs, image_shape = (720., 1280.), max_boxes=10, score_threshold=.6, iou_threshold=.5):
    """
    将YOLO编码的输出（很多盒子）连同它们的分数，盒子坐标和类一起转换为预测的盒子。

    参数:
    yolo_outputs -- output of the encoding model (for image_shape of (608, 608, 3)), contains 4 tensors:
                    box_confidence: tensor of shape (None, 19, 19, 5, 1)
                    box_xy: tensor of shape (None, 19, 19, 5, 2)
                    box_wh: tensor of shape (None, 19, 19, 5, 2)
                    box_class_probs: tensor of shape (None, 19, 19, 5, 80)
    image_shape -- tensor of shape (2,) containing the input shape, in this notebook we use (608., 608.) (has to be float32 dtype)
    max_boxes -- integer, maximum number of predicted boxes you'd like
    score_threshold -- real value, if [ highest class probability score < threshold], then get rid of the corresponding box
    iou_threshold -- real value, "intersection over union" threshold used for NMS filtering

    返回值:
    scores -- tensor of shape (None, ), predicted score for each box
    boxes -- tensor of shape (None, 4), predicted box coordinates
    classes -- tensor of shape (None,), predicted class for each box
    """

    ### START CODE HERE ###

    # 检索YOLO模型的输出 (≈1 line)
    box_confidence, box_xy, box_wh, box_class_probs = yolo_outputs

    # 转换框以准备进行过滤功能
    boxes = yolo_boxes_to_corners(box_xy,box_wh)

    # 使用已实现的功能之一以阈值score_threshold执行分数过滤 (≈1 line)
    scores, boxes, classes = yolo_filter_boxes(box_confidence, boxes, box_class_probs, score_threshold)

    # 将框缩放回原始图像形状。
    boxes = yolo_utils.scale_boxes(boxes, image_shape)

    # 使用已实现的功能之一，以iou_threshold阈值执行非最大抑制 (≈1 line)
    scores, boxes, classes = yolo_non_max_suppression(scores, boxes, classes, max_boxes, iou_threshold)

    ### END CODE HERE ###

    return scores, boxes, classes
```

In [11]:

```
#测试yolo_eval

with tf.Session() as test_b:
    yolo_outputs = (tf.random_normal([19, 19, 5, 1], mean=1, stddev=4, seed = 1),
                    tf.random_normal([19, 19, 5, 2], mean=1, stddev=4, seed = 1),
                    tf.random_normal([19, 19, 5, 2], mean=1, stddev=4, seed = 1),
                    tf.random_normal([19, 19, 5, 80], mean=1, stddev=4, seed = 1))
    scores, boxes, classes = yolo_eval(yolo_outputs)
    print("scores[2] = " + str(scores[2].eval()))
    print("boxes[2] = " + str(boxes[2].eval()))
    print("classes[2] = " + str(classes[2].eval()))
    print("scores.shape = " + str(scores.eval().shape))
    print("boxes.shape = " + str(boxes.eval().shape))
    print("classes.shape = " + str(classes.eval().shape))
```

```
scores[2] = 138.79124
boxes[2] = [1292.3297  -278.52167 3876.9893  -835.56494]
classes[2] = 54
scores.shape = (10,)
boxes.shape = (10, 4)
classes.shape = (10,)
```

练习：实现模型预测方法 yolo_predict

In [17]:

```
def predict(sess, image_file):
    """
    运行存储在" sess"中的图形以预测" image_file"的框。 打印并绘制预测值。

    参数:
    sess -- 包含YOLO图的tensorflow / Keras会话
    image_file -- 存储在"images"文件夹中的图像的名称

    返回值:
    out_scores -- 形状张量（None，），预测盒的分数
    out_boxes -- 形状的张量（None，4），预测框的坐标
    out_classes -- 形状张量（None，），预测框的类索引

    Note: " None"实际上代表预测的盒子数，在0到max_boxes之间变化。
    """

    # Preprocess your image
    image, image_data = preprocess_image("images/" + image_file, model_image_size = (608, 608))

    # 使用正确的张量运行会话，并在feed_dict中选择正确的占位符。
    # feed_dict={yolo_model.input: ... , K.learning_phase(): 0})
    ### START CODE HERE ### (≈ 1 line)
    out_scores, out_boxes, out_classes = sess.run([scores, boxes, classes], feed_dict = {yolo_model.input:image_data, K.learning_phas

    ### END CODE HERE ###

    # Print predictions info
    print('Found {} boxes for {}'.format(len(out_boxes), image_file))
    # Generate colors for drawing bounding boxes.
    colors = generate_colors(class_names)
    # Draw bounding boxes on the image file
    draw_boxes(image, out_scores, out_boxes, out_classes, class_names, colors)
    # Save the predicted bounding box on the image
    image.save(os.path.join("out", image_file), quality=90)
    # Display the results in the notebook
    output_image = scipy.misc.imread(os.path.join("out", image_file))
    imshow(output_image)

    return out_scores, out_boxes, out_classes
```

In [18]:

```
# 在 tset.jpg 上进行测试
out_scores, out_boxes, out_classes = predict(sess, "test.jpg")
```

```
Found 7 boxes for test.jpg
car 0.60 (925, 285) (1045, 374)
car 0.66 (706, 279) (786, 350)
bus 0.67 (5, 266) (220, 407)
car 0.70 (947, 324) (1280, 705)
car 0.74 (159, 303) (346, 440)
car 0.80 (761, 282) (942, 412)
car 0.89 (367, 300) (745, 648)
```

```
/opt/conda/lib/python3.6/site-packages/ipykernel_launcher.py:36: DeprecationWarning: `imread` is deprecated!
`imread` is deprecated in SciPy 1.0.0, and will be removed in 1.2.0.
Use ``imageio.imread`` instead.
```

人脸识别与神经风格迁移

练习：实现三元组损失函数

In [4]:

```
# GRADED FUNCTION: triplet_loss

def triplet_loss(y_true, y_pred, alpha = 0.2):
    """
    Implementation of the triplet loss as defined by formula (3)

    Arguments:
    y_true -- true labels, required when you define a loss in Keras, you don't need it in this function.
    y_pred -- python list containing three objects:
            anchor -- the encodings for the anchor images, of shape (None, 128)
            positive -- the encodings for the positive images, of shape (None, 128)
            negative -- the encodings for the negative images, of shape (None, 128)

    Returns:
    loss -- real number, value of the loss
    """

    anchor, positive, negative = y_pred[0], y_pred[1], y_pred[2]

    ### START CODE HERE ### (≈ 4 lines)
    # Step 1: Compute the (encoding) distance between the anchor and the positive, you will need to sum over axis=-1
    pos_dist = tf.reduce_sum(tf.square(tf.subtract(anchor,positive)),axis=-1)
    # Step 2: Compute the (encoding) distance between the anchor and the negative, you will need to sum over axis=-1
    neg_dist = tf.reduce_sum(tf.square(tf.subtract(anchor,negative)),axis=-1)
    # Step 3: subtract the two previous distances and add alpha.
    basic_loss = tf.add(tf.subtract(pos_dist,neg_dist),alpha)
    # Step 4: Take the maximum of basic_loss and 0.0. Sum over the training examples.
    loss = tf.reduce_sum(tf.maximum(basic_loss,0))
    ### END CODE HERE ###

    return loss
```

In [5]:

```
#测试triplet_loss

with tf.Session() as test:
    tf.set_random_seed(1)
    y_true = (None, None, None)
    y_pred = (tf.random_normal([3, 128], mean=6, stddev=0.1, seed = 1),
              tf.random_normal([3, 128], mean=1, stddev=1, seed = 1),
              tf.random_normal([3, 128], mean=3, stddev=4, seed = 1))
    loss = triplet_loss(y_true, y_pred)

    print("loss = " + str(loss.eval()))
```

```
loss = 528.1426
```

练习：实现 verify() 方法

In [9]:
```python
# GRADED FUNCTION: verify

def verify(image_path, identity, database, model):
    """
    Function that verifies if the person on the "image_path" image is "identity".

    Arguments:
    image_path -- path to an image
    identity -- string, name of the person you'd like to verify the identity. Has to be a resident of the Happy house.
    database -- python dictionary mapping names of allowed people's names (strings) to their encodings (vectors).
    model -- your Inception model instance in Keras

    Returns:
    dist -- distance between the image_path and the image of "identity" in the database.
    door_open -- True, if the door should open. False otherwise.
    """

    ### START CODE HERE ###

    # Step 1: Compute the encoding for the image. Use img_to_encoding() see example above. (≈ 1 line)
    encoding = fr_utils.img_to_encoding(image_path, model)

    # Step 2: Compute distance with identity's image (≈ 1 line)
    dist = np.linalg.norm(encoding - database[identity])

    # Step 3: Open the door if dist < 0.7, else don't open (≈ 3 lines)
    if dist < 0.7:
        print("欢迎 " + str(identity) + "回家！")
        is_door_open = True
    else:
        print("经验证，您与" + str(identity) + "不符！")
        is_door_open = False

    ### END CODE HERE ###

    return dist, is_door_open
```

Younes 想进来，然后门上摄像头拍了一张图片("images/camera_0.jpg")，然我们验证一下吧。

[10]:
```python
#测试verity
verify("images/camera_0.jpg", "younes", database, FRmodel)
```

欢迎 younes回家！

[10]: (0.67100716, True)

期望输出:

```
It's younes, welcome home!

(0.65939283, True)
```

Benoit 上周打破了鱼缸，已经被赶出去了，数据库也除名了，他拿了 Kian 的ID卡想进来试图说自己就是Kian，门上摄像头采集看他一张照片("images/camera_2.jpg")，让我们验证一下吧。

[11]:
```python
verify("images/camera_2.jpg", "kian", database, FRmodel)
```

经验证，您与kian不符！

[11]: (0.8580015, False)

练习：实现 who_is_it()

```python
def who_is_it(image_path, database, model):
    """
    Implements face recognition for the happy house by finding who is the person on the image_path image.

    Arguments:
    image_path — path to an image
    database — database containing image encodings along with the name of the person on the image
    model — your Inception model instance in Keras

    Returns:
    min_dist — the minimum distance between image_path encoding and the encodings from the database
    identity — string, the name prediction for the person on image_path
    """

    ### START CODE HERE ###

    ## Step 1: Compute the target "encoding" for the image. Use img_to_encoding() see example above. ## (≈ 1 line)
    encoding = fr_utils.img_to_encoding(image_path, model)

    ## Step 2: Find the closest encoding ##
    min_dist = 100
    # Initialize "min_dist" to a large value, say 100 (≈1 line)


    # Loop over the database dictionary's names and encodings.
    for (name, db_enc) in database.items():

        # Compute L2 distance between the target "encoding" and the current "emb" from the database. (≈ 1 line)
        dist = np.linalg.norm(encoding - db_enc)

        # If this distance is less than the min_dist, then set min_dist to dist, and identity to name. (≈ 3 lines)
        if dist < min_dist:
            min_dist = dist
            identity = name

    ### END CODE HERE ###

    if min_dist > 0.7:
        print("Not in the database.")
    else:
        print ("it's " + str(identity) + ", the distance is " + str(min_dist))

    return min_dist, identity
```

Younes 走到门前，摄像头采集了他的一张照片（"images/camera_0.jpg"），让我们使用 who_it_is() 来试试吧。

```python
In [14]: #测试who_it_is
         who_is_it("images/camera_0.jpg", database, FRmodel)

         it's younes, the distance is 0.67100716

Out[14]: (0.67100716, 'younes')
```

词向量的运算与 Emoji 生成器

实现余弦相似度计算

```python
In [5]: def cosine_similarity(u, v):
            """
            u与v的余弦相似度反映了u与v的相似程度

            参数:
                u -- 维度为(n,)的词向量
                v -- 维度为(n,)的词向量

            返回:
                cosine_similarity -- 由上面公式定义的u和v之间的余弦相似度。
            """
            distance = 0

            ###在这里填入代码###

            # 计算u与v的内积
            dot = np.dot(u, v)

            #计算u的L2范数
            norm_u = np.sqrt(np.sum(np.power(u, 2)))

            #计算v的L2范数
            norm_v = np.sqrt(np.sum(np.power(v, 2)))

            # 根据公式1计算余弦相似度
            cosine_similarity = np.divide(dot, norm_u * norm_v)

            ###在这里填入代码###

            return cosine_similarity
```

In [6]:
```
#测试cosine_similarity
print("==============测试cosine_similarity==============")
father = word_to_vec_map["father"]
mother = word_to_vec_map["mother"]
ball = word_to_vec_map["ball"]
crocodile = word_to_vec_map["crocodile"]
france = word_to_vec_map["france"]
italy = word_to_vec_map["italy"]
paris = word_to_vec_map["paris"]
rome = word_to_vec_map["rome"]

print("cosine_similarity(father, mother) = ", cosine_similarity(father, mother))
print("cosine_similarity(ball, crocodile) = ",cosine_similarity(ball, crocodile))
print("cosine_similarity(france - paris, rome - italy) = ",cosine_similarity(france - paris, rome - italy))
```

```
==============测试cosine_similarity==============
cosine_similarity(father, mother) =  0.8909038442893615
cosine_similarity(ball, crocodile) =  0.2743924626137942
cosine_similarity(france - paris, rome - italy) =  -0.6751479308174201
```

词类类比

In [8]:
```
def complete_analogy(word_a, word_b, word_c, word_to_vec_map):
    """
    解决"A与B相比就类似于C与___相比一样"之类的问题

    参数:
        word_a — 一个字符串类型的词
        word_b — 一个字符串类型的词
        word_c — 一个字符串类型的词
        word_to_vec_map — 字典类型，单词到GloVe向量的映射

    返回:
        best_word — 满足(v_b - v_a) 最接近 (v_best_word - v_c) 的词
    """
    ###在这里填入代码###

    # 把单词转换为小写
    word_a, word_b, word_c = word_a.lower(), word_b.lower(), word_c.lower()

    # 获取对应单词的词向量
    e_a, e_b, e_c = word_to_vec_map[word_a], word_to_vec_map[word_b], word_to_vec_map[word_c]

    # 获取全部的单词
    words = word_to_vec_map.keys()

    # 将max_cosine_sim初始化为一个比较大的负数
    max_cosine_sim = -100
    best_word = None

    # 遍历整个数据集
    for word in words:
        # 要避免匹配到输入的数据
        if word in [word_a, word_b, word_c]:
            continue
        # 计算余弦相似度
        cosine_sim = cosine_similarity((e_b - e_a), (word_to_vec_map[word] - e_c))

        if cosine_sim > max_cosine_sim:
            max_cosine_sim = cosine_sim
            best_word = word

    ###在这里填入代码###

    return best_word
```

In [9]:
```
#测试complete_analogy
print("==============测试complete_analogy==============")
triads_to_try = [('italy', 'italian', 'spain'), ('india', 'delhi', 'japan'), ('man', 'woman', 'boy'), ('small', 'smaller', 'large')]
for triad in triads_to_try:
    print ('{} -> {} <====> {} -> {}'.format( *triad, complete_analogy(*triad,word_to_vec_map)))
```

```
==============测试complete_analogy==============
italy -> italian <====> spain -> spanish
india -> delhi <====> japan -> tokyo
man -> woman <====> boy -> girl
small -> smaller <====> large -> larger
```

实现 Emojifier-V1

In [25]:
```python
def sentence_to_avg(sentence, word_to_vec_map):
    """
    将句子转换为单词列表，提取其GloVe向量，然后将其平均。

    参数:
        sentence — 字符串类型，从X中获取的样本。
        word_to_vec_map — 字典类型，单词映射到50维的向量的字典

    返回:
        avg — 对句子的均值编码，维度为(50,)
    """
    ###在这里填入代码###

    # 第一步：分割句子，转换为列表。
    words = sentence.lower().split()

    # 初始化均值词向量
    avg = np.zeros(50,)

    # 第二步：对词向量取平均。
    for w in words:
        avg += word_to_vec_map[w]
    avg = np.divide(avg, len(words))

    ###在这里填入代码###

    return avg
```

In [26]:
```python
#测试sentence_to_avg
print("==========测试sentence_to_avg==========")
avg = sentence_to_avg("Morrocan couscous is my favorite dish", word_to_vec_map)
print("avg = ", avg)
```

```
==========测试sentence_to_avg==========
avg =  [-0.008005    0.56370833 -0.50427333  0.258865    0.55131103  0.03104983
 -0.21013718  0.16893933 -0.09590267  0.141784   -0.15708967  0.18525867
  0.6495785   0.38371117  0.21102167  0.11301667  0.02613967  0.26037767
  0.05820667 -0.01578167 -0.12078833 -0.02471267  0.4128455   0.5152061
  0.38756167 -0.898661   -0.535145    0.33501167  0.68806933 -0.2156265
  1.797155    0.10476933 -0.36775333  0.750785    0.10282583  0.348925
 -0.27262833  0.66768    -0.10706167 -0.283635    0.59580117  0.28747333
 -0.3366635   0.23393817  0.34349183  0.178405    0.1166155  -0.076433
  0.1445417   0.09808667]
```

In [32]:
```python
def model(X, Y, word_to_vec_map, learning_rate=0.01, num_iterations=400):
    """
    在numpy中训练词向量模型。

    参数:
        X — 输入的字符串类型的数据，维度为(m, 1)。
        Y — 对应的标签，0-7的数组，维度为(m, 1)。
        word_to_vec_map — 字典类型的单词到50维词向量的映射。
        learning_rate — 学习率.
        num_iterations — 迭代次数。

    返回:
        pred — 预测的向量，维度为(m, 1)。
        W — 权重参数，维度为(n_y, n_h)。
        b — 偏置参数，维度为(n_y,)
    """
    np.random.seed(1)

    # 定义训练数量
    m = Y.shape[0]
    n_y = 5
    n_h = 50

    # 使用Xavier初始化参数
    W = np.random.randn(n_y, n_h) / np.sqrt(n_h)
    b = np.zeros((n_y,))

    # 将Y转换成独热编码
    Y_oh = emo_utils.convert_to_one_hot(Y, C=n_y)

    # 优化循环
    for t in range(num_iterations):
        for i in range(m):
            ###在这里填入代码###

            # 获取第i个训练样本的均值
            avg = sentence_to_avg(X[i], word_to_vec_map)

            # 前向传播
            z = np.dot(W, avg) + b
            a = emo_utils.softmax(z)

            # 计算第i个训练的损失
            cost = -np.sum(Y_oh[i]*np.log(a))
```

```
            # 计算第i个训练的损失
            cost = -np.sum(Y_oh[i]*np.log(a))

            # 计算梯度
            dz = a - Y_oh[i]
            dW = np.dot(dz.reshape(n_y,1), avg.reshape(1, n_h))
            db = dz

            # 更新参数
            W = W - learning_rate * dW
            b = b - learning_rate * db

            ###在这里填入代码###

        if t % 100 == 0:
            print("第{t}轮，损失为{cost}".format(t=t,cost=cost))
            pred = emo_utils.predict(X, Y, W, b, word_to_vec_map)

    return pred, W, b
```

```
print(X_train.shape)
print(Y_train.shape)
print(np.eye(5)[Y_train.reshape(-1)].shape)
print(X_train[0])
print(type(X_train))
Y = np.asarray([5,0,0,5, 4, 4, 4, 6, 6, 4, 1, 1, 5, 6, 6, 3, 6, 3, 4, 4])
print(Y.shape)

X = np.asarray(['I am going to the bar tonight', 'I love you', 'miss you my dear',
 'Lets go party and drinks','Congrats on the new job','Congratulations',
 'I am so happy for you', 'Why are you feeling bad', 'What is wrong with you',
 'You totally deserve this prize', 'Let us go play football',
 'Are you down for football this afternoon', 'Work hard play harder',
 'It is suprising how people can be dumb sometimes',
 'I am very disappointed','It is the best day in my life',
 'I think I will end up alone','My life is so boring','Good job',
 'Great so awesome'])
```

```
(132,)
(132,)
(132, 5)
never talk to me again
```

```
In [34]: #训练
         pred, W, b = model(X_train, Y_train, word_to_vec_map)

         第0轮，损失为1.9520498812810076
         Accuracy: 0.3484848484848485
         第100轮，损失为0.07971818726014794
         Accuracy: 0.9318181818181818
         第200轮，损失为0.04456369243681402
         Accuracy: 0.9545454545454546
         第300轮，损失为0.03432267378786059
         Accuracy: 0.9696969696969697
```

实现 pretrained_embedding_layer()函数

```
In [41]: def pretrained_embedding_layer(word_to_vec_map, word_to_index):
    """
    创建Keras Embedding()层，加载已经训练好了的50维GloVe向量

    参数：
        word_to_vec_map — 字典类型的单词与词嵌入的映射
        word_to_index — 字典类型的单词到词汇表（400,001个单词）的索引的映射。

    返回：
        embedding_layer() — 训练好了的Keras的实体层。
    """
    vocab_len = len(word_to_index) + 1
    emb_dim = word_to_vec_map["cucumber"].shape[0]

    ###在这里填入代码###

    # 初始化嵌入矩阵
    emb_matrix = np.zeros((vocab_len, emb_dim))

    # 将嵌入矩阵的每行的“index”设置为词汇“index”的词向量表示
    for word, index in word_to_index.items():
        emb_matrix[index, :] = word_to_vec_map[word]

    # 定义Keras的embbeding层
    embedding_layer = Embedding(vocab_len, emb_dim, trainable=False)

    # 构建embedding层。
    embedding_layer.build((None,))

    # 将嵌入层的权重设置为嵌入矩阵。
    embedding_layer.set_weights([emb_matrix])

    ###在这里填入代码###

    return embedding_layer
```

```
In [43]: #测试pretrained_embedding_layer
         print("==============测试pretrained_embedding_layer==============")
         embedding_layer = pretrained_embedding_layer(word_to_vec_map, word_to_index)
         print("weights[0][1][3] =", embedding_layer.get_weights()[0][1][3])

         ==============测试pretrained_embedding_layer==============
         weights[0][1][3] = -0.3403
```

构建 Emojifier-V2

```
In [44]: def Emojify_V2(input_shape, word_to_vec_map, word_to_index):
    """
    实现Emojify-V2模型的计算图

    参数：
        input_shape — 输入的维度，通常是(max_len,)
        word_to_vec_map — 字典类型的单词与词嵌入的映射。
        word_to_index — 字典类型的单词到词汇表（400,001个单词）的索引的映射。

    返回：
        model — Keras模型实体
    """

    ###在这里填入代码###

    # 定义sentence_indices为计算图的输入，维度为(input_shape,)，类型为dtype 'int32'
    sentence_indices = Input(input_shape, dtype='int32')

    # 创建embedding层
    embedding_layer = pretrained_embedding_layer(word_to_vec_map, word_to_index)

    # 通过嵌入层传播sentence_indices，你会得到嵌入的结果
    embeddings = embedding_layer(sentence_indices)

    # 通过带有128维隐藏状态的LSTM层传播嵌入
    # 需要注意的是，返回的输出应该是一批序列。
    X = LSTM(128, return_sequences=True)(embeddings)
    # 使用dropout，概率为0.5
    X = Dropout(0.5)(X)
    # 通过另一个128维隐藏状态的LSTM层传播X
    # 注意，返回的输出应该是单个隐藏状态，而不是一组序列。
    X = LSTM(128, return_sequences=False)(X)
    # 使用dropout，概率为0.5
    X = Dropout(0.5)(X)
    # 通过softmax激活的Dense层传播X，得到一批5维向量。
    X = Dense(5)(X)
    # 添加softmax激活
    X = Activation('softmax')(X)

    # 创建模型实体
    model = Model(inputs=sentence_indices, outputs=X)

    ###在这里填入代码###

    return model
```

机器翻译与触发词检测

练习：实现 one_step_attention()

In [6]:
```
# 将共享层定义为全局变量
repeator = RepeatVector(Tx)
concatenator = Concatenate(axis=-1)
densor1 = Dense(10, activation = "tanh")
densor2 = Dense(1, activation = "relu")
activator = Activation(softmax, name='attention_weights') # 在这个 notebook 我们正在使用自定义的 softmax(axis = 1)
dotor = Dot(axes = 1)
```

```
WARNING: Logging before flag parsing goes to stderr.
W1130 08:56:55.773979 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:74: The name tf.get_default_graph is deprecated. Please use tf.compat.v1.get_default_graph instead.
```

现在您可以使用这些图层来实现 one_step_attention() 。为了通过这些层之一传播Keras张量对象 X ，使用 layer(X) (或 layer([X,Y]) 如果它需要多个输入)。

例如， densor（X） 将通过上面定义的 Dense（1） 层传播 X 。

In [7]:
```
###在这里填入代码###
# GRADED FUNCTION: one_step_attention

def one_step_attention(a, s_prev):
    """
    执行一步 attention: 输出一个上下文向量，输出作为注意力权重的点积计算的上下文向量
    "alphas" Bi-LSTM的 隐藏状态 "a"

    参数:
    a — Bi-LSTM的输出隐藏状态 numpy-array 维度 (m, Tx, 2*n_a)
    s_prev — (post-attention) LSTM的前一个隐藏状态, numpy-array 维度(m, n_s)

    返回:
    context — 上下文向量, 下一个(post-attetion) LSTM 单元的输入
    """

    # 使用 repeator 重复 s_prev 维度 (m, Tx, n_s) 这样你就可以将它与所有隐藏状态"a" 连接起来。 (≈ 1 line)
    s_prev = repeator(s_prev)
    # 使用 concatenator 在最后一个轴上连接 a 和 s_prev (≈ 1 line)
    concat = concatenator([a, s_prev])
    # 使用 densor1 传入参数 concat, 通过一个小的全连接神经网络来计算“中间能量”变量 e。(≈1 lines)
    e = densor1(concat)
    # 使用 densor2 传入参数 e , 通过一个小的全连接神经网络来计算“能量”变量 energies。(≈1 lines)
    energies = densor2(e)
    # 使用 activator 传入参数 "energies" 计算注意力权重 "alphas" (≈ 1 line)
    alphas = activator(energies)
    # 使用 dotor 传入参数 "alphas" 和 "a" 计算下一个 ((post-attention) LSTM 单元的上下文向量 (≈ 1 line)
    context = dotor([alphas, a])

    return context
```

练习：实现 model()

In [8]:
```
n_a = 32
n_s = 64
post_activation_LSTM_cell = LSTM(n_s, return_state = True)
output_layer = Dense(len(machine_vocab), activation=softmax)
```

In [9]:
```python
###在这里填入代码###
# GRADED FUNCTION: model

def model(Tx, Ty, n_a, n_s, human_vocab_size, machine_vocab_size):
    """
    参数:
    Tx -- 输入序列的长度
    Ty -- 输出序列的长度
    n_a -- Bi-LSTM的隐藏状态大小
    n_s -- post-attention LSTM的隐藏状态大小
    human_vocab_size -- python字典 "human_vocab" 的大小
    machine_vocab_size -- python字典 "machine_vocab" 的大小

    返回:
    model -- Keras 模型实例
    """

    # 定义模型的输入，维度 (Tx,)
    # 定义 s0 和 c0, 初始化解码器 LSTM 的隐藏状态，维度 (n_s,)
    X = Input(shape=(Tx, human_vocab_size))
    s0 = Input(shape=(n_s,), name='s0')
    c0 = Input(shape=(n_s,), name='c0')
    s = s0
    c = c0

    # 初始化一个空的输出列表
    outputs = []


    # 第一步：定义 pre-attention Bi-LSTM。 记得使用 return_sequences=True. (≈ 1 line)
    a = Bidirectional(LSTM(n_a, return_sequences=True), input_shape=(m, Tx, n_a * 2))(X)

    # 第二步：迭代 Ty 步
    for t in range(Ty):

        # 第二步.A: 执行一步注意机制，得到在 t 步的上下文向量 (≈ 1 line)
        context = one_step_attention(a, s)

        # 第二步.B: 使用 post-attention LSTM 单元得到新的 "context"
        # 别忘了使用： initial_state = [hidden state, cell state] (≈ 1 line)
        s, _, c = post_activation_LSTM_cell(context, initial_state=[s, c])

        # 第二步.C: 使用全连接层处理post-attention LSTM 的隐藏状态输出 (≈ 1 line)
        out = output_layer(s)

        # 第二步.D: 追加 "out" 到 "outputs" 列表 (≈ 1 line)
        outputs.append(out)

    # 第三步：创建模型实例，获取三个输入并返回输出列表。 (≈ 1 line)
    model = Model(inputs=[X, s0, c0], outputs=outputs)

    return model
```

运行以下单元以创建模型。

In [10]:
```python
model = model(Tx, Ty, n_a, n_s, len(human_vocab), len(machine_vocab))
```

```
W1130 09:00:04.494677 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:517: The name tf.placeholder is deprecated. Please use tf.compat.v1.placeholder instead.

W1130 09:00:04.685090 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:4138: The name tf.random_uniform is deprecated. Please use tf.random.uniform instead.
```

让我们查看模型的摘要，以检查它是否与预期输出相匹配。

In [11]:
```python
model.summary()
```

```
                                                                 dot_1[9][0]
                                                                 lstm_1[8][0]
                                                                 lstm_1[8][2]
__________________________________________________________________________________________________
dense_3 (Dense)                 (None, 11)           715         lstm_1[0][0]
                                                                 lstm_1[1][0]
                                                                 lstm_1[2][0]
                                                                 lstm_1[3][0]
                                                                 lstm_1[4][0]
                                                                 lstm_1[5][0]
                                                                 lstm_1[6][0]
                                                                 lstm_1[7][0]
                                                                 lstm_1[8][0]
                                                                 lstm_1[9][0]
==================================================================================================
Total params: 52,960
Trainable params: 52,960
Non-trainable params: 0
__________________________________________________________________________________________________
```

像往常一样，在Keras创建模型后，你需要编译它并定义模型使用的损失，优化和评估指标。

编译模型时，损失使用 `categorical_crossentropy`，优化算法使用 Adam optimizer（`learning rate = 0.005`，$\beta_1 = 0.9$，$\beta_2 = 0.999$，`decay = 0.01`），评估指标使用 ['accuracy']:

In [12]:
```
###在这里填入代码### (≈2 lines)
# 定义优化器
opt = Adam(lr=0.005, beta_1=0.9, beta_2=0.999, decay=0.01)
# 模型编译
model.compile(loss='categorical_crossentropy', optimizer=opt, metrics=['accuracy'])
###在这里填入代码###
```

```
W1130 09:00:28.936295 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/optimizers.py:790: The name t
f.train.Optimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.

W1130 09:00:28.951699 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:3295: The name tf.log is deprecated. Please use tf.math.log instead.
```

In [13]:
```
s0 = np.zeros((m, n_s))
c0 = np.zeros((m, n_s))
outputs = list(Yoh.swapaxes(0,1))
```

现在我们训练模型，迭代一次

In [14]:
```
model.fit([Xoh, s0, c0], outputs, epochs=1, batch_size=100)
```

```
W1130 09:00:34.250859 140617741920064 deprecation.py:323] From /opt/conda/lib/python3.6/site-packages/tensorflow_core/python/ops/math_grad.p
y:1424: where (from tensorflow.python.ops.array_ops) is deprecated and will be removed in a future version.
Instructions for updating:
Use tf.where in 2.0, which has the same broadcast rule as np.where
W1130 09:00:41.345285 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:986: The name tf.assign_add is deprecated. Please use tf.compat.v1.assign_add instead.

W1130 09:00:41.502468 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:973: The name tf.assign is deprecated. Please use tf.compat.v1.assign instead.

W1130 09:00:41.663582 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:2741: The name tf.Session is deprecated. Please use tf.compat.v1.Session instead.

W1130 09:00:41.668800 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:174: The name tf.get_default_session is deprecated. Please use tf.compat.v1.get_default_session instead.

W1130 09:00:41.669666 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:181: The name tf.ConfigProto is deprecated. Please use tf.compat.v1.ConfigProto instead.
```

```
Epoch 1/1
```

```
W1130 09:00:41.885312 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:190: The name tf.global_variables is deprecated. Please use tf.compat.v1.global_variables instead.

W1130 09:00:41.886826 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:199: The name tf.is_variable_initialized is deprecated. Please use tf.compat.v1.is_variable_initialized instead.

W1130 09:00:42.986345 140617741920064 module_wrapper.py:139] From /opt/conda/lib/python3.6/site-packages/keras/backend/tensorflow_backend.p
y:206: The name tf.variables_initializer is deprecated. Please use tf.compat.v1.variables_initializer instead.
```

```
10000/10000 [==============================] - 17s 2ms/step - loss: 16.7787 - dense_3_loss: 2.5869 - dense_3_acc: 0.5623 - dense_3_acc_1: 0.
6963 - dense_3_acc_2: 0.3021 - dense_3_acc_3: 0.0921 - dense_3_acc_4: 0.9613 - dense_3_acc_5: 0.2943 - dense_3_acc_6: 0.0414 - dense_3_acc_
7: 0.9179 - dense_3_acc_8: 0.2798 - dense_3_acc_9: 0.1013
```

运行下面的单元加载我们的权重。（通过几万个模型训练，您应该能够获得类似精度的模型，但加载我们的模型将节省您的时间。）

In [15]:
```
model.load_weights('./ex13data/models/model.h5')
```

In [16]:
```
EXAMPLES = ['3 May 1979', '5 April 09', '21th of August 2016', 'Tue 10 Jul 2007', 'Saturday May 9 2018', 'March 3 2001', 'March 3rd 2001', '1
for example in EXAMPLES:

    source = string_to_int(example, Tx, human_vocab)
    source = np.array(list(map(lambda x: to_categorical(x, num_classes=len(human_vocab)), source)))
    source = np.expand_dims(source, axis=0)
    prediction = model.predict([source, s0, c0])
    prediction = np.argmax(prediction, axis = -1)
    output = [inv_machine_vocab[int(i)] for i in prediction]

    print("source:", example)
    print("output:", ''.join(output))
```

```
source: 3 May 1979
output: 1979-05-03
source: 5 April 09
output: 2009-05-05
source: 21th of August 2016
output: 2016-08-21
source: Tue 10 Jul 2007
output: 2007-07-10
source: Saturday May 9 2018
output: 2018-05-09
source: March 3 2001
output: 2001-03-03
source: March 3rd 2001
output: 2001-03-03
source: 1 March 2001
output: 2001-03-01
```

五、实验总结

通过本次实验，我了解了如何使用 YOLO 模型进行识别和定位车辆、如何完成 Happy House 的人脸识别、学习了词向量的运算以及 Emoji 生成器和机器翻译，扩充了我的知识，并且运行了一些相关项目，让我对这些知识更加熟悉。